

Control and Visualization of an Android-Based Autonomous Navigation System Using Dead Reckoning and SLAM

Jainil Patel Rishi Patel Aaryan Sinh Shaurya
202201030@dau.ac.in 202201037@dau.ac.in 202201075@dau.ac.in

Sujal Manavadariya Denil Antala
202201084@dau.ac.in 202201090@dau.ac.in

Abstract—Autonomous navigation relies heavily on accurate localization and mapping techniques for real-time control and decision-making. This project presents the design and implementation of an Android-based autonomous navigation system integrating Dead Reckoning (DR) using inertial measurement unit (IMU) sensors and Simultaneous Localization and Mapping (SLAM) using ARCore. The system visualizes real-time trajectories of both DR and SLAM for comparative analysis. A Kalman filter-based estimation framework is explored for minimizing sensor drift. The developed application demonstrates the feasibility of smartphone-based autonomous navigation and provides an experimental platform for analyzing estimation errors, drift behavior, and control accuracy. A video demonstration of the working system is also provided.

Index Terms—Autonomous Systems, Dead Reckoning, SLAM, ARCore, IMU Sensors, Kalman Filtering, Android Navigation

I. INTRODUCTION

Autonomous systems require precise estimation of position and orientation in unknown environments to perform navigation and control tasks effectively. Smartphones today contain high-quality inertial sensors and cameras that enable low-cost implementation of navigation algorithms.

Dead Reckoning (DR) estimates position from IMU integration but suffers from drift. SLAM corrects this error using visual features. This project integrates both approaches on Android using ARCore.

II. PROBLEM STATEMENT AND OBJECTIVES

A. Problem Statement

To design and implement a smartphone-based autonomous navigation system capable of estimating and visualizing real-time motion trajectories using DR and SLAM.

B. Objectives

- Implement IMU-based Dead Reckoning.
- Integrate ARCore-based SLAM.
- Visualize real-time trajectories.
- Compare drift effects.

III. SYSTEM ARCHITECTURE

The overall system architecture of the proposed autonomous navigation framework is shown in Figure 1. The system consists of three main modules: Dead Reckoning, SLAM, and Visualization and Control.

The Dead Reckoning module processes inertial data from the smartphone's accelerometer, gyroscope, and magnetometer. After noise filtering and calibration, the data is integrated to estimate position and orientation without using any GPS or external location services.

In parallel, the SLAM module utilizes the smartphone camera and inertial sensors through the ARCore framework to estimate the six degrees-of-freedom pose using visual feature tracking and mapping.

The outputs of both localization methods are forwarded to a common analysis and visualization layer, where trajectory deviation and drift are evaluated in real time. A session logging module stores motion data for offline analysis and performance evaluation.

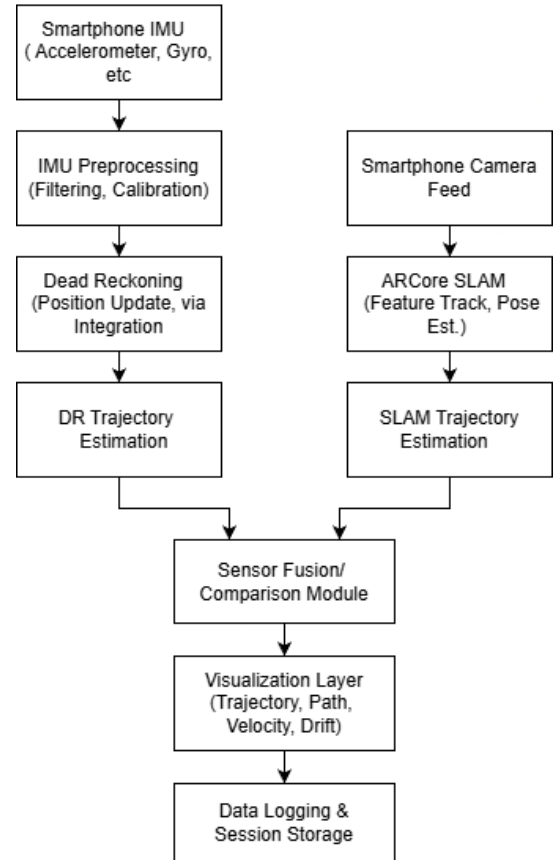


Fig. 1. System architecture

IV. METHODOLOGY

The proposed system estimates the motion of a smartphone-based autonomous platform using two parallel localization techniques: Dead Reckoning (DR) based on inertial sensors and Simultaneous Localization and Mapping (SLAM) using ARCore. The overall methodology consists of sensor data acquisition, preprocessing, motion estimation, trajectory generation, visualization, and performance comparison.

A. System Workflow

The smartphone collects inertial measurements from the accelerometer, gyroscope, and magnetometer, along with camera frames for SLAM-based pose estimation. The Dead Reckoning and SLAM algorithms operate in parallel, and their estimated trajectories are compared and visualized in real time. Session data is stored for offline analysis. Notably, no GPS or external location services are used in the system.

B. Inertial Sensor Data Acquisition

The smartphone's Inertial Measurement Unit (IMU) provides:

- Linear acceleration: $\mathbf{a}(t) = [a_x, a_y, a_z]^T$
- Angular velocity: $\boldsymbol{\omega}(t) = [\omega_x, \omega_y, \omega_z]^T$
- Magnetic field vector: $\mathbf{m}(t) = [m_x, m_y, m_z]^T$

These signals are sampled at discrete time intervals t_k and are affected by sensor noise and bias.

C. Preprocessing and Calibration

To reduce noise and systematic errors, sensor preprocessing is performed before motion estimation. A low-pass filter is applied to accelerometer data to remove high-frequency noise, while gyroscope bias is estimated and compensated. Gravity components are removed using orientation estimates from sensor fusion. This preprocessing ensures improved stability in the Dead Reckoning computations.

D. Dead Reckoning Motion Estimation

Dead Reckoning estimates the position of the device by integrating acceleration over time. The velocity is computed as:

$$\mathbf{v}_k = \mathbf{v}_{k-1} + \mathbf{a}_k \Delta t \quad (1)$$

The position is then updated as:

$$\mathbf{p}_k = \mathbf{p}_{k-1} + \mathbf{v}_k \Delta t \quad (2)$$

where $\mathbf{a}_k$, $\mathbf{v}_k$, and $\mathbf{p}_k$ represent the acceleration, velocity, and position at time step k , and Δt is the sampling period. Since Dead Reckoning relies entirely on inertial integration, errors accumulate over time, resulting in drift.

E. SLAM Using ARCore

The SLAM module uses the smartphone camera and inertial sensors through ARCore. Visual features are extracted from incoming frames and tracked over time. These features are used to estimate the device pose:

$$\mathbf{x}_k = [x_k, y_k, z_k, \theta_k, \phi_k, \psi_k]^T \quad (3)$$

where the vector represents position and orientation in three-dimensional space. ARCore internally performs visual-inertial fusion to correct drift and maintain robust localization even during temporary sensor disturbances.

F. Trajectory Comparison and Visualization

The DR and SLAM trajectories are projected onto a common 2D coordinate frame for real-time visualization. Separate colored paths are used to distinguish the two motion estimates. Velocity, drift magnitude, and trajectory deviation are displayed on the mobile interface to evaluate the performance difference between the two localization methods.

G. Session Logging and Data Storage

All estimated positions, velocities, timestamps, and drift values are stored locally during each navigation session. This recorded data is used for offline analysis, result plotting, and system validation.

H. Sensor Fusion Using Kalman Filtering

To reduce noise and correct drift in the Dead Reckoning estimates, a Kalman filtering approach is studied as part of the sensor fusion process. The Kalman filter combines the inertial predictions from the DR model with corrective pose observations obtained from the SLAM module.

The system state is represented as:

$$\mathbf{x}_k = [x_k, y_k, v_{x_k}, v_{y_k}]^T \quad (4)$$

The prediction step is governed by:

$$\mathbf{x}_k^- = \mathbf{A}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k \quad (5)$$

and the measurement update is given by:

$$\mathbf{x}_k = \mathbf{x}_k^- + \mathbf{K}_k(\mathbf{z}_k - \mathbf{H}\mathbf{x}_k^-) \quad (6)$$

where $\mathbf{z}_k$ represents the SLAM-based pose measurement and $\mathbf{K}_k$ is the Kalman gain. This fusion improves localization accuracy by balancing fast inertial updates with visually corrected pose information.

V. IMPLEMENTATION

The proposed system was implemented using Android Studio with native Android APIs and the ARCore SDK. The inertial sensor data was accessed using the Android Sensor Manager, while real-time camera-based tracking was achieved through ARCore. Dead Reckoning computations were performed using processed IMU data, and SLAM-based pose estimates were obtained directly from ARCore. Both trajectories were visualized in real time on a custom Android user interface. The application also supports session data storage for offline analysis.

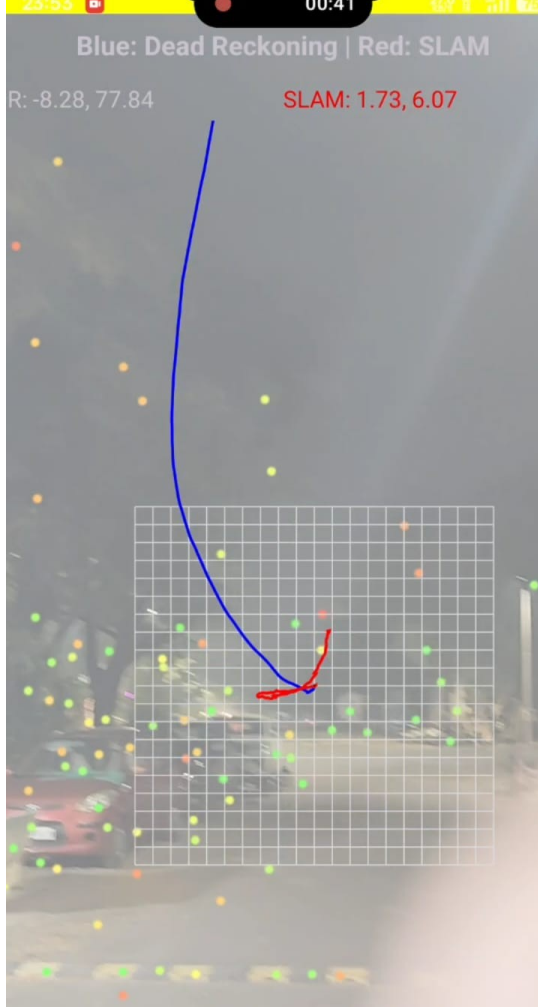


Fig. 2. Android UI with real-time trajectory

VI. RESULTS

The experimental results demonstrate a clear difference between Dead Reckoning (DR) and SLAM-based localization. As observed in the captured trajectories, the DR path (shown in blue) initially follows the motion but gradually deviates due to cumulative integration errors, leading to noticeable drift. In contrast, the SLAM trajectory (shown in red) closely follows the actual motion path with better stability and consistency. The SLAM-based approach benefits from continuous visual feature tracking and pose correction, resulting in significantly reduced drift compared to Dead Reckoning.

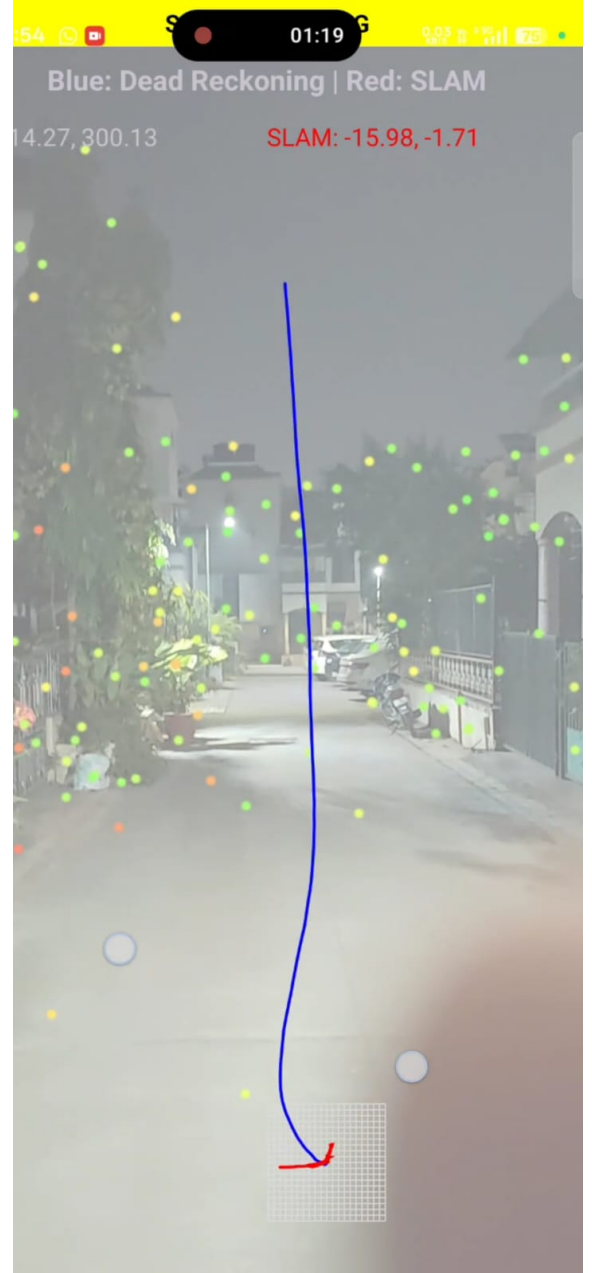


Fig. 3. Dead Reckoning trajectory



Fig. 4. SLAM-based trajectory

SLAM provides significantly better stability and lower drift than Dead Reckoning.

A. Video Demonstration

The working demonstration is available in the submitted project video.

VII. CHALLENGES AND LIMITATIONS

- **Drift in IMU Integration:** The Dead Reckoning method relies on double integration of acceleration data, which leads to cumulative errors over time. Even small sensor biases result in significant positional drift during long motion sequences.

- **Hardware Dependency of ARCore:** The performance of the SLAM module is highly dependent on the smartphone's camera quality, processing power, and sensor accuracy. Older or low-end devices may experience reduced tracking stability and frame drops.
- **Sensor Noise and Environmental Disturbances:** Inertial sensors are affected by measurement noise and vibrations, while the SLAM system is sensitive to poor lighting conditions, motion blur, and lack of distinct visual features, which can degrade localization accuracy.

VIII. DELIVERABLES

- **Video Demonstration:** [Click here to view](#)
- **Source Code (ZIP File):** [Click here to download](#)

IX. FUTURE WORK

- Extended Kalman Filters.
- Persistent environment mapping.
- Multi-agent SLAM.

X. CONCLUSION

The project successfully demonstrates a hybrid autonomous navigation system using Android. SLAM greatly improves localization accuracy over pure Dead Reckoning.

REFERENCES

- [1] H. Durrant-Whyte and T. Bailey, "Simultaneous Localization and Mapping," IEEE Robotics and Automation Magazine.
- [2] Google, "ARCore Developer Documentation."
- [3] D. Titterton and J. Weston, "Strapdown Inertial Navigation Technology," IET.